Database Lab : Multi-Tier Blocking Lock Resolution & Governance SOP

Duration : 60 Min.

Scenario

As a Lead Database Reliability Engineer at NIIT, a cascading transaction lock chain has paralyzed core payment processing. Over 50 downstream sessions are blocked in a 3-tier hierarchy. You must interrogate `pg_locks`, isolate the single top-level root blocker PID, safely terminate it using `pg_terminate_backend`, and establish lock timeout governance.

Input Details

The environment has been pre-configured with the following local database platform instances:

- **Database Host:** `localhost` (Ubuntu Linux VM)
- **PostgreSQL Service:** `localhost:5432` | DB: `postgres` | User: `postgres` | Password: `PostgresPassword123!` | CLI: `psql`

Username & Naming Conventions

Your candidate prefix is: **`<prefix>`** (e.g. `student-exam123`).

Resource naming: - **Lock SOP Table:** `<prefix>_multitier_lock_sop`

Task Objectives

Perform the following configuration and lock resolution steps via `psql`:

1. Interrogate Lock Hierarchy

- Connect via `psql` to `localhost:5432`.
- Query `pg_locks` and `pg_stat_activity` to inspect lock dependencies.

2. Create Lock SOP Table

- Create table `<prefix>_multitier_lock_sop` with columns:
- `sop_id SERIAL PRIMARY KEY`
- `root_blocker_pid INT NOT NULL`
- `intermediate_pids VARCHAR(100)`
- `resolution_command TEXT NOT NULL`
- `lock_timeout_setting VARCHAR(30) NOT NULL`
- `resolved_by VARCHAR(50) NOT NULL`

3. Terminate Blocker & Configure Statement Timeout

- Execute `SELECT pg_terminate_backend(<root_pid>);` on any active blocking session.
- Insert resolution details into `<prefix>_multitier_lock_sop` with `lock_timeout_setting = 'lock_timeout=10s'` and `resolved_by = '<prefix>'`.
- Set session lock timeout: `SET lock_timeout = '10s';`.

Verification

Once completed, run the verification script:

```
python run.py
```

The driver connects to PostgreSQL, asserts that lock contention is resolved, and verifies the SOP table.

Grading Criteria

Your performance will be evaluated based on the following test cases:

Test Case	Requirement	Marks
TC1	Catalog Inspection & Lock Tree Traversal (pg_locks)	3 Marks
TC2	Root Blocker Identification & Waiter Exclusion	3 Marks
TC3	Lock SOP Table (_multitier_lock_sop) Creation	3 Marks
TC4	Safe Termination Command Formulation (pg_terminate_backend)	3 Marks
TC5	Cascading Waiter Release Verification	2 Marks
TC6	Lock Timeout Configuration (lock_timeout=10s)	2 Marks
TC7	Post-Incident Governance Metadata Documentation	2 Marks
TC8	Live Database Concurrency State Sign-Off	2 Marks

Total Score: 20 Marks

Important Notes

- Replace `<prefix>` with your candidate prefix.
- All actions must be executed on the live PostgreSQL instance.